

Advanced Prompting & Structured Outputs

1. Objective

The objective of this task is to understand how advanced prompting techniques can make Large Language Model (LLM) responses more controlled, consistent, and useful for software applications.

The main concepts explored are:

- Zero-shot prompting
- Few-shot prompting
- Providing examples
- Separating instructions from context
- Structured outputs
- JSON responses
- Output consistency
- Prompt improvement

2. Zero-Shot Prompting

Definition

Zero-shot prompting means asking an AI model to perform a task without providing examples of how the task should be performed.

Initial Prompt

Classify each customer message as Positive, Negative, or Neutral.

Messages:

1. "I absolutely love the new product. It works perfectly!"
2. "My order arrived two days late and I am disappointed."
3. "Can you tell me when my order will arrive?"

4. "The support team was very helpful and solved my issue."
5. "The product is okay, but nothing special."

Observation

The model can perform the task without examples because the categories are clearly defined. However, for more complicated classification tasks, the model may interpret categories differently.

3. Few-Shot Prompting

Definition

Few-shot prompting means providing the AI model with examples before asking it to perform the task.

Improved Prompt

You are a customer message classifier.

Classify customer messages into exactly one of these categories:

- Positive
- Negative
- Neutral

Use the following examples:

Example 1:

Customer: "I love this product. It is excellent!"

Classification: Positive

Example 2:

Customer: "My order is late and I am very disappointed."

Classification: Negative

Example 3:

Customer: "Can you tell me when my package will arrive?"

Classification: Neutral

Now classify these messages:

1. "The new product is fantastic and works perfectly."
2. "I have been waiting for my order for a week."
3. "What payment methods do you accept?"
4. "Your support team solved my problem quickly."
5. "The product is acceptable, but I expected more."

4. Separating Instructions and Context

A good prompt can separate **instructions** from the **context/data** that the model needs to process.

Example

INSTRUCTIONS:

Classify the customer message.

Return the category and priority.

Use only the allowed categories and priorities.

ALLOWED CATEGORIES:

Sales

Support

Complaint

General

ALLOWED PRIORITIES:

Low

Medium

High

CONTEXT:

Customer message:

"My order has not arrived even though it was supposed to arrive three days ago."

This makes the prompt easier to understand and reduces ambiguity.

5. Structured Outputs

A normal AI response might look like this:

The customer is complaining because their order has been delayed. This is a high-priority support issue because the expected delivery date has already passed.

This is understandable to a human, but software has difficulty reliably extracting individual fields.

Instead, we can request:

```
{  
  "customer_name": "Ali",  
  "issue": "Delayed order",  
  "order_status": "Delayed",  
  "priority": "High"  
}
```

Now each piece of information is stored in a predictable field.

6. Customer Message Extraction

Customer Message

Hello, my name is Ali Khan. I ordered a laptop five days ago, but it still has not arrived. The expected delivery date was yesterday. Please check the status of my order.

Prompt

Extract the following information from the customer message:

- customer_name
- issue
- order_status
- priority

Return the result as JSON.

Allowed priority values:
Low, Medium, High

Customer message:

"Hello, my name is Ali Khan. I ordered a laptop five days ago, but it still has not arrived. The expected delivery date was yesterday. Please check the status of my order."

Structured Result

```
{  
  "customer_name": "Ali Khan",  
  "issue": "Order has not arrived",  
  "order_status": "Delayed",  
  "priority": "High"  
}
```

7. Practical Task: AI Customer Message Classifier

Initial Prompt

Classify the following customer message as Sales, Support, Complaint, or General.

Also determine the priority as Low, Medium, or High.

Give a short reason for your classification.

Customer message:

"My package has not arrived yet and it was supposed to arrive three days ago."

Example Result

Category: Complaint

Priority: High

Reason: The customer's order is significantly delayed and they are requesting assistance.

The problem is that this output is written as normal text and may not always have exactly the same format.

8. Improved Structured Prompt

ROLE:

You are an AI customer message classification system.

TASK:

Analyze the customer message and classify it.

INSTRUCTIONS:

1. Identify the customer's intent.
2. Classify the message into exactly one category.
3. Determine the priority.
4. Provide a short reason.
5. Return ONLY valid JSON.
6. Do not add explanations outside the JSON object.

ALLOWED CATEGORIES:

- Sales
- Support
- Complaint
- General

ALLOWED PRIORITIES:

- Low
- Medium
- High

CATEGORY GUIDELINES:

- Sales: Questions about products, pricing, purchasing, or availability.
- Support: Requests for technical or account assistance.
- Complaint: Customer dissatisfaction, service problems, delays, or negative experiences.
- General: General questions that do not fit the other categories.

PRIORITY GUIDELINES:

- Low: General information or non-urgent requests.
- Medium: A problem requiring attention but not immediately urgent.
- High: Serious problems, significant delays, payment problems, or urgent customer issues.

REQUIRED JSON FORMAT:

```
{  
  "category": "Sales | Support | Complaint | General",  
  "priority": "Low | Medium | High",  
  "reason": "Short explanation"  
}
```

CUSTOMER MESSAGE:

```
{{customer_message}}
```

9. Test Message 1

Input

I want to buy your premium laptop. Can you tell me the price and whether it is currently available?

Structured Result

```
{  
  "category": "Sales",  
  "priority": "Low",  
  "reason": "The customer is interested in purchasing a product and wants pricing and  
availability information."  
}
```

10. Test Message 2

Input

My account is not allowing me to log in. I have tried resetting my password several times but it still does not work.

Structured Result

```
{  
  "category": "Support",  
  "priority": "Medium",  
  "reason": "The customer is experiencing an account login problem and needs technical  
assistance."  
}
```

11. Test Message 3

Input

I ordered my package five days ago and it was supposed to arrive three days ago. I still haven't received it. This is unacceptable.

Structured Result

```
{  
  "category": "Complaint",  
  "priority": "High",  
  "reason": "The customer is dissatisfied because their order is significantly delayed."  
}
```

13. Why JSON Is Useful in Software Applications

A paragraph is primarily designed for **human readers**, while structured JSON can be designed for **software systems**.

For example:

```
{  
  "category": "Complaint",  
  "priority": "High",  
  "reason": "Order significantly delayed"  
}
```

A backend application can easily read:

category → Complaint
priority → High
reason → Order significantly delayed

The application could then automatically:

- Create a support ticket
- Set ticket priority to High
- Send the ticket to the complaints department
- Notify a customer-support employee
- Store the information in a database
- Trigger an automation workflow

This is particularly useful in **customer support, CRM systems, recruitment systems, document processing, email automation, and AI agents**.
